How-To Library

Informatica

How to Verify Addresses, Email Addresses, and Phone Numbers for Custom Objects in Spring 2019 Version 6.60

© Copyright Informatica LLC 2015, 2019. Informatica LLC. No part of this document may be reproduced or transmitted in any form, by any means (electronic, photocopying, recording or otherwise) without prior consent of Informatica LLC. All other company and product names may be trade names or trademarks of their respective owners and/or copyrighted materials of such owners.

Abstract

To verify addresses, email addresses, and phone numbers of the custom objects in Salesforce, you must create custom Visualforce pages for custom objects. You must also embed the attributes of the Informatica Data as a Service (DaaS) component into the custom Visualforce pages. This article shows how to create the custom object creation and the DaaS editor Visualforce pages, create a controller for the custom object, and create a trigger for the custom object.

Supported Versions

- Informatica Cloud Customer 360 Spring 2019 Version.6.60

Table of Contents

Overview. 2
Prerequisites for Verifying Addresses, Email Addresses, and Phone Numbers of Custom Objects. 3
Step 1: Create a Trigger for the Custom Object. 3
Sample Apex Code. 4
Step 2: Create a Visualforce Page to Create a Custom Object. 4
Sample Apex Code. 6
Step 3: Create a Controller for the DaaS Editor Visualforce Page. 8
Sample Apex Code. 10
Step 4: Create a DaaS Editor Visualforce Page for the Custom Object. 11
Sample Apex Code. 13
Step 5: Override the Standard Salesforce Object Creation Page with Visualforce Page. 15
Verifying Addresses, Email Addresses, and Phone Numbers. 16

Overview

You can verify addresses, email addresses, and phone numbers of standard and custom objects in Salesforce. For standard objects, you can verify the contact information on the account, contact, and lead scout pages. For custom objects, you must create custom Visualforce pages and embed the DaaS component into the custom Visualforce pages. The custom Visualforce pages are the custom object creation Visualforce page and the DaaS editor Visualforce page.

You must create a trigger for the custom object. A trigger is an Apex code that executes before or after you perform any of the following operations on the custom object:

- Insert
- Update
- Delete
- Merge
- Upsert
- Undelete

After you create the trigger, create the custom Visualforce pages. You must also create a controller for the custom pages to specify what happens when a user interacts with the components in the Visualforce page.

For example, if you have a custom object called `Employee__c`, you can create a trigger, named `EmployeeTrigger` to perform custom actions before or after any changes to `Employee__c`. You can create custom Visualforce pages called `EmployeeScoutPage` for custom object creation and `EmployeePage` which is the DaaS editor page. You can embed the attributes of the DaaS component such as `FieldGroupName` and `Edit` to the custom pages. You can create a controller named `EmployeeController` to specify what happens when the user clicks the Edit button in the DaaS editor Visualforce page.

Prerequisites for Verifying Addresses, Email Addresses, and Phone Numbers of Custom Objects

To verify addresses, email addresses, and phone numbers of custom objects, you must perform the following tasks:

1. Create a trigger for the custom object.
2. Create a Visualforce page to create a custom object.
3. Create a controller for the DaaS editor Visualforce page.
4. Create a DaaS editor Visualforce page for the custom object.
5. Override the standard Salesforce object creation page with Visualforce page.

Step 1: Create a Trigger for the Custom Object

You must create a trigger for the custom object. Apex triggers enable you to perform custom actions before or after changes to the Salesforce records, such as insertions, updates, or deletions. The trigger code is stored as metadata under the custom object.

1. Select your name and click **Setup**.
2. Under **App Setup**, select **Create** > **Objects**.

   The **Custom Objects** page appears.
3. Click the custom object.

   The **Custom Object Definition Detail** page appears.
4. In the **Triggers** section, click **New**.

   The **Apex Trigger Edit** page appears.
5. On the **Version Settings** tab, select the installed package as Cloud Customer 360.
6. To compile and enable the trigger, select **Is Active**.

   By default, **Is Active** is selected.
7. In the trigger editor, enter the Apex code in the following format to create a trigger for the custom object:

   ```
   trigger <Trigger Name> on <Custom Object Name> (after delete, after insert, after
   undelete, after update, before delete, before insert, before update)
   {
   DSE.API_CustomScoutClass cs = new DSE.API_CustomScoutClass('<Custom Object Name>');
   cs.triggerSynchronize(trigger.new, trigger.old, Trigger.isInsert, Trigger.isUpdate,
   Trigger.isDelete, Trigger.isUnDelete, Trigger.isBefore, Trigger.isAfter);
   }
   ```

   The Apex code format uses the following parameters:

   - `Custom Object Name`. Name of the custom object for which you create the trigger.
   - `Trigger Name`. Name of the trigger that you create.
8. Click **Save**.

Sample Apex Code

The following sample Apex code shows how you can create a trigger named `EmpTrigger` for the custom object `Employee__c`:

```
trigger EmpTrigger on Employee__c (after delete, after insert, after undelete, after update,
before delete, before insert, before update)
{
    DSE.API_CustomScoutClass cs = new DSE.API_CustomScoutClass('Employee__c');

    cs.triggerSynchronize(trigger.new, trigger.old, Trigger.isInsert,
    Trigger.isUpdate, Trigger.isDelete, Trigger.isUnDelete, Trigger.isBefore, Trigger.isAfter);
}
```

Step 2: Create a Visualforce Page to Create a Custom Object

After you create a trigger for the custom object, you must create a custom object creation Visualforce page and embed the DaaS components.

1. Select your name and click **Setup**.
2. Under **App Setup**, select **Develop** > **Visualforce Pages**.

   The **Visualforce Pages** page appears.
3. Click **New**.

   The Visualforce edit page appears.
4. In the **Page Information** section, perform the following tasks:
   1. Type a label for the custom object creation Visualforce page.
   2. Type a name for the Visualforce page.

      Note: By default, the name of the Visualforce page is the same as the label.
   3. Optionally, edit the description of the Visualforce page.
5. On the **Visualforce Markup** tab, enter the Apex code in the following format to create a Visualforce page to create a custom object:

```
<apex:page sidebar="true" showheader="true" standardcontroller="
    <Custom Object Name>">
        <apex:stylesheet value="{!URLFOR($Resource.DSE__bootstrap_sf1, 'bootstrap_sf1/
bootstrap-sf1-0.1.0-beta.15/dist/css/bootstrap-namespaced.css')}" />
        <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery-ui.css')}" />
        <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery.js')}" />
        <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery-ui.min.js')}" />
        <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery.blockUI.js')}" />
        <Style>
        .bootstrap-sf1 .card ul.card-detail.card-detail-labels > li .card-detail-value
{text-align: left;}
        .bootstrap-sf1 .form-group {margin-bottom: 3px;color: #000;}
        .bootstrap-sf1 .form-control, .bootstrap-sf1 .form-group{font-size :12px;}
        .bootstrap-sf1 label{font-size :11px;color: #4a4a56;}
        .bootstrap-sf1 .form-group input.form-control, .bootstrap-sf1 .form-group
select.form-control{height: 30px;}
        .bootstrap-sf1 textarea {resize: vertical;}
        .center {text-align: center;}
        .bootstrap-sf1 input[type="checkbox"]{width:auto !important;height:auto !
important;}
        .requiredInput {position: relative;height: 100%;}
        .requiredBlock {background-color: #c00;position: absolute;left: -4px;width:
3px;top: 1px;bottom: 1px;}
```

```
        .lookupInput>input, .dateInput>input{display:-webkit-inline-box !important;width:
100%!important;font-size: 14px !important;}
        .lookupInput>a, .dateInput>a{width: auto !important;display: contents !
important;height: 34px !important;background-color: none !important;}
        tr.multiSelectPicklistRow> td> select {width:auto !important;}
        input[type="text"]{
            display: block;
            width: 100%;
            height: 30px;
            padding: 4px 4px;
            font-size: 14px;
            line-height: 1.42857143;
            color: #000000;
            background-color: #ffffff;
            background-image: none;
            border: 1px solid #cfd0d2;
            border-radius: 4px;
            -webkit-box-shadow: inset 0 1px 1px rgba(0, 0, 0, 0.075);
            box-shadow: inset 0 1px 1px rgba(0, 0, 0, 0.075);
            -webkit-transition: border-color ease-in-out 0.15s, box-shadow ease-in-out
0.15s;
            -o-transition: border-color ease-in-out 0.15s, box-shadow ease-in-out 0.15s;
            transition: border-color ease-in-out 0.15s, box-shadow ease-in-out 0.15s;
        }
    </Style>
        <apex:form id="frm">
            <div class="bootstrap-sf1" id="wrapper">
                <div class="panel-body">
                    <div class="center">
                        <apex:commandbutton value="
                            <Button Label for Save>" action="{!save}" styleClass="btn
btn-default btn-xs"/>
                        </div>
                        <div class="form-group">
                            <div class="row">
                                <div class="col-lg-2">
                                    <apex:outputLabel value="Name" />
                                </div>
                                <div class="col-lg-2 dropdown col-sm-10 col-xs-10 col-
md-10">
                                    <apex:inputField value="{!
                                        <Custom Object Name>.name}" >
                                        </apex:inputField>
                                </div>
                            </div>
                            <!-- DaaS Fields -->
                            <div class="form-group">
                                <div class="row">
                                    <div class="col-lg-6">
                                        <DSE:DS_DataQuality RecordRef="{!
                                            <Custom Object Name>}" FieldGroupName="
                                                <DaaS Mapping Name1>"
FormGroupStyle="form-group" RowStyle="row" LabelStyle="col-lg-4" InputStyle="col-lg-4
col-sm-10 col-xs-10 col-md-10" StatusStyle="col-lg-1 col-sm-1 col-xs-1 col-md-1"
FormControlStyle="form-control" Edit="true" />
                                        </div>
                                    </div>
                                </div>
                                <!-- DaaS Fields -->
                                <div class="form-group">
                                    <div class="row">
                                        <div class="col-lg-6">
                                            <DSE:DS_DataQuality RecordRef="{!
                                                <Custom Object Name>}"
FieldGroupName="
                                                    <DaaS Mapping Name2>"
FormGroupStyle="form-group" RowStyle="row" LabelStyle="col-lg-4" InputStyle="col-lg-4
col-sm-10 col-xs-10 col-md-10" StatusStyle="col-lg-1 col-sm-1 col-xs-1 col-md-1"
```

```
    FormControlStyle="form-control" Edit="true" />
                                                    </div>
                                                </div>
                                            </div>
                                            <!-- DaaS Fields -->
                                            <div class="form-group">
                                                <div class="row">
                                                    <div class="col-lg-6">
                                                        <DSE:DS_DataQuality RecordRef="{!
                                                            <Custom Object Name>}"
    FieldGroupName="
                                                                <DaaS Mapping Name3>"
    FormGroupStyle="form-group" RowStyle="row" LabelStyle="col-lg-4" InputStyle="col-lg-4
    col-sm-10 col-xs-10 col-md-10" StatusStyle="col-lg-1 col-sm-1 col-xs-1 col-md-1"
    FormControlStyle="form-control" Edit="true" />
                                                    </div>
                                                </div>
                                            </div>
                                        </div>
                                    </apex:form>
                                </apex:page>
```

The Apex code format uses the following parameters:

- `Custom Object Name`. Name of the custom object for which you create a Visualforce page.
- `Button Label for Save`. Label of the button that performs the save operation.
- `DaaS Mapping Name1`, `DaaS Mapping Name2`, and `DaaS Mapping Name3`. Field group names that you specify when you create DaaS mappings.
- `DS_DataQuality` attributes. You can configure the following attributes:
  - RecordRef. Name of the custom object for which you create the Visualforce page.
  - FieldGroupName. Name of the DaaS mapping used in the Visualforce page.
  - FormGroupStyle. Salesforce bootstrap style for form groups.
  - RowStyle. Salesforce bootstrap style for rows.
  - LabelStyle. Column widths for the labels. You can specify column widths for small, medium, and large screens.
  - InputStyle. Column widths for the input fields. The column widths set the appearance of the input fields on small, medium, and large screens.
  - StatusStyle. Column widths for the verification status. The column widths set the appearance of the verification status on small, medium, and large screens.
  - FormControlStyle. Salesforce bootstrap style for the input fields.
  - Edit. Boolean value that indicates whether the input fields are editable or not. Set to true to make the input fields editable, and set to false to make the input fields non-editable.

6. Click **Save**.

Sample Apex Code

The following sample Apex code creates a Visualforce page named `EmployeeScoutPage` and embeds the DaaS components for the field groups named `Employee Address`, `Employee Email`, and `Employee Phone`:

```
<apex:page sidebar="true" showheader="true" standardcontroller="Employee__c">
    <apex:stylesheet value="{!URLFOR($Resource.DSE__bootstrap_sf1, 'bootstrap_sf1/bootstrap-
sf1-0.1.0-beta.15/dist/css/bootstrap-namespaced.css')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery-
ui.css')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery.js')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery-
ui.min.js')}" />
```

```
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery.blockUI.js')}" />
    <style>
        .bootstrap-sf1 .card ul.card-detail.card-detail-labels > li .card-detail-value {text-
align: left;}
        .bootstrap-sf1 .form-group {margin-bottom: 3px;color: #000;}
        .bootstrap-sf1 .form-control, .bootstrap-sf1 .form-group{font-size :12px;}
        .bootstrap-sf1 label{font-size :11px;color: #4a4a56;}
        .bootstrap-sf1 .form-group input.form-control, .bootstrap-sf1 .form-group select.form-
control{height: 30px;}
        .bootstrap-sf1 textarea {resize: vertical;}
        .center {text-align: center;}
        .bootstrap-sf1 input[type="checkbox"]{width:auto !important;height:auto !important;}
        .requiredInput {position: relative;height: 100%;}
        .requiredBlock {background-color: #c00;position: absolute;left: -4px;width: 3px;top:
1px;bottom: 1px;}
        .lookupInput>input, .dateInput>input{display:-webkit-inline-box !important;width:100%!
important;font-size: 14px !important;}
        .lookupInput>a, .dateInput>a{width: auto !important;display: contents !
important;height: 34px !important;background-color: none !important;}
        tr.multiSelectPicklistRow> td> select {width:auto !important;}
        input[type="text"]{
            display: block;
            width: 100%;
            height: 30px;
            padding: 4px 4px;
            font-size: 14px;
            line-height: 1.42857143;
            color: #000000;
            background-color: #ffffff;
            background-image: none;
            border: 1px solid #cfd0d2;
            border-radius: 4px;
            -webkit-box-shadow: inset 0 1px 1px rgba(0, 0, 0, 0.075);
            box-shadow: inset 0 1px 1px rgba(0, 0, 0, 0.075);
            -webkit-transition: border-color ease-in-out 0.15s, box-shadow ease-in-out 0.15s;
            -o-transition: border-color ease-in-out 0.15s, box-shadow ease-in-out 0.15s;
            transition: border-color ease-in-out 0.15s, box-shadow ease-in-out 0.15s;
        }
    </style>
    <apex:form id="frm">
        <div class="bootstrap-sf1" id="wrapper">
            <div class="panel-body">
                <div class="center">
                    <apex:commandbutton value="Create Employee" action="{!save}"
styleClass="btn btn-default btn-xs"/>
                </div>
                <div class="form-group">
                    <div class="row">
                        <div class="col-lg-2">
                            <apex:outputLabel value="Name" />
                        </div>
                        <div class="col-lg-2 dropdown col-sm-10 col-xs-10 col-md-10">
                            <apex:inputField value="{!Employee__c.name}"></apex:inputField>
                        </div>
                    </div>
                </div>
                <!-- DaaS Fields -->
                <div class="form-group">
                    <div class="row">
                        <div class="col-lg-6">
                            <DSE:DS_DataQuality
                                    RecordRef="{!Employee__c}"
                                    FieldGroupName="Employee Address"
                                    FormGroupStyle="form-group"
                                    RowStyle="row"
                                    LabelStyle="col-lg-4"
                                    InputStyle="col-lg-4 col-sm-10 col-xs-10 col-md-10"
                                    StatusStyle="col-lg-1 col-sm-1 col-xs-1 col-md-1"
                                    FormControlStyle="form-control"
                                    Edit="true"
```

```
                        />
                    </div>
                </div>
            </div>
            <!-- DaaS Fields -->
            <!-- DaaS Fields -->
            <div class="form-group">
                <div class="row">
                    <div class="col-lg-6">
                        <DSE:DS_DataQuality
                            RecordRef="{!Employee__c}"
                            FieldGroupName="Employee Phone"
                            FormGroupStyle="form-group"
                            RowStyle="row"
                            LabelStyle="col-lg-4"
                            InputStyle="col-lg-4 col-sm-10 col-xs-10 col-md-10"
                            StatusStyle="col-lg-1 col-sm-1 col-xs-1 col-md-1"
                            FormControlStyle="form-control"
                            Edit="true"
                        />
                    </div>
                </div>
            </div>
            <!-- DaaS Fields -->
            <!-- DaaS Fields -->
            <div class="form-group">
                <div class="row">
                    <div class="col-lg-6">
                        <DSE:DS_DataQuality
                                RecordRef="{!Employee__c}"
                                FieldGroupName="Employee Email"
                                FormGroupStyle="form-group"
                                RowStyle="row"
                                LabelStyle="col-lg-4"
                                InputStyle="col-lg-4 col-sm-10 col-xs-10 col-md-10"
                                StatusStyle="col-lg-1 col-sm-1 col-xs-1 col-md-1"
                                FormControlStyle="form-control"
                                Edit="true"
                            />
                    </div>
                </div>
            </div>
            <!-- DaaS Fields -->
        </div>
    </div>
</apex:form>
</apex:page>
```

Step 3: Create a Controller for the DaaS Editor Visualforce Page

Before you create the DaaS editor Visualforce page, you must create a controller for the Visualforce page. A controller is an Apex class that specifies what happens when a user interacts with the components specified in the associated Visualforce markup. User interactions can be the click of a button or a link. Use controllers to access the data that you want to display in a page or to change component behavior.

1. Select your name and click **Setup**.
2. Under **App Setup**, select **Develop** > **Apex Classes**.

   The **Apex Classes** page appears.
3. Click **New**.

   The **Apex Class Edit** page appears.
4. On the **Version Settings** tab, select the installed package as Cloud Customer 360.
5. Click **Apex Class**.

6. In the class editor, enter the Apex code in the following format to create a controller for a Visualforce page:

```
public with sharing class <Controller Name> {
    public Static String s{get;set;}
    public <Custom Object Name> <Variable>{get;set;}
    public String[] input{get;set;}
    public String[] output{get;set;}
    public Boolean mode{get;set;}
    public String wrapperStyle{get;set;}
    public String labelStyle{get;set;}
    public String statusStyle{get;set;}
    public String inputStyle{get;set;}
    public class applicationException extends Exception {}

    public <Controller Name>(Apexpages.StandardController std){
        String <Field> = Apexpages.currentPage().getParameters().get('id');
        if(String.isNotBlank(<Field>)){
            List<<Custom Object Name>> ac = [Select <Field1>,<Field2>,...,<FieldN>, from
<Custom Object Name> where <Condition>];

            if(!ac.isempty())
                a = ac[0];
        }else{
            throw new applicationException('Please provide valid Id while trying to edit
record');
        }
        mode = true;
        wrapperStyle = 'col-lg-8 col-md-8 col-sm-8 col-xs-12';
        labelStyle = 'col-lg-4';
        inputStyle = 'col-lg-9 col-md-9 col-sm-9 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
    }

    public Pagereference save(){
        upsert a;
        mode = false;
        wrapperStyle = 'col-lg-8 col-md-8 col-sm-8 col-xs-12';
        labelStyle = 'col-lg-4';
        inputStyle = 'col-lg-9 col-md-9 col-sm-9 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
        return null;
    }

    public Pagereference editMode(){
        mode = true;
        wrapperStyle = 'col-lg-12 col-md-12 col-sm-12 col-xs-12';
        labelStyle = 'col-lg-4 col-md-4 col-sm-4 col-xs-8 right';
        inputStyle = 'col-lg-6 col-md-6 col-sm-6 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
        return null;
    }

    public Pagereference readMode(){
        mode = false;
        wrapperStyle = 'col-lg-8 col-md-8 col-sm-8 col-xs-12';
        labelStyle = 'col-lg-4';
        inputStyle = 'col-lg-9 col-md-9 col-sm-9 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
        return null;
    }
}
```

The Apex code format uses the following parameters:

- `Controller Name`. Name of the controller that you create for the `EmployeePage`.
- `Custom Object Name`. Name of the custom object for which you create a controller.
- `Condition`. Condition that you specify based on which you retrieve the records.

- `Field`. Field that you use in `Condition` based on which you retrieve the records.
- `Field1, Field2,..., and FieldN`. Fields that you retrieve from the custom object.
- `Variable`. Variable that references the records. Use the same variable when you create a DaaS editor Visualforce page for the custom object.

7. Click **Save**.

Sample Apex Code

The following sample Apex code creates a controller named `EmployeeController`:

```
public with sharing class EmployeeController {
    public Static String s{get;set;}
    public Employee__c a{get;set;}
    public String[] input{get;set;}
    public String[] output{get;set;}
    public Boolean mode{get;set;}
    public String wrapperStyle{get;set;}
    public String labelStyle{get;set;}
    public String statusStyle{get;set;}
    public String inputStyle{get;set;}
    public class applicationException extends Exception {}

    public EmployeeController(Apexpages.StandardController std){
        String empID = Apexpages.currentPage().getParameters().get('id');
                if(String.isNotBlank(empID)){
            List<Employee__c> ac = [Select id,Name, Street__c, State__c,
Country__c ,Email__c,Phone__c,DAAS_Addr_AddressCode_Type__c,
            DAAS_Phone_StatusDescription__c,City__c,DAAS_Email_StatusDescription__c from
Employee__c where Id =:empID];

            if(!ac.isempty())
                a = ac[0];
        }else{
            throw new applicationException('Please provide valid Id while trying to edit
record');
        }
        mode = true;
        wrapperStyle = 'col-lg-8 col-md-8 col-sm-8 col-xs-12';
        labelStyle = 'col-lg-4';
        inputStyle = 'col-lg-9 col-md-9 col-sm-9 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
    }

    public Pagereference save(){
        upsert a;
        mode = false;
        wrapperStyle = 'col-lg-8 col-md-8 col-sm-8 col-xs-12';
        labelStyle = 'col-lg-4';
        inputStyle = 'col-lg-9 col-md-9 col-sm-9 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
        return null;
    }

    public Pagereference editMode(){
        mode = true;
        wrapperStyle = 'col-lg-12 col-md-12 col-sm-12 col-xs-12';
        labelStyle = 'col-lg-4 col-md-4 col-sm-4 col-xs-8 right';
        inputStyle = 'col-lg-6 col-md-6 col-sm-6 col-xs-9';
        statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
        return null;
    }

    public Pagereference readMode(){
        mode = false;
        wrapperStyle = 'col-lg-8 col-md-8 col-sm-8 col-xs-12';
        labelStyle = 'col-lg-4';
```

```
            inputStyle = 'col-lg-9 col-md-9 col-sm-9 col-xs-9';
            statusStyle = 'col-lg-1 col-md-1 col-sm-1 col-xs-1';
            return null;
        }
    }
```

Step 4: Create a DaaS Editor Visualforce Page for the Custom Object

After you create a controller, you must create a DaaS editor Visualforce page. You can embed the DaaS component in the DaaS editor Visualforce page.

1. Select your name and click **Setup**.
2. Under **App Setup**, select **Develop** > **Visualforce Pages**.

   The **Visualforce Pages** page appears.
3. Click **New**.

   The Visualforce edit page appears.
4. In the **Page Information** section, perform the following tasks:
   1. Type a label for the DaaS editor Visualforce page.
   2. Type a name for the Visualforce page.

      Note: By default, the name of the Visualforce page is the same as the label.
   3. Optionally, edit the description of the Visualforce page.
5. On the **Visualforce Markup** tab, enter the Apex code in the following format to create a custom object DaaS editor Visualforce page:

```
<apex:page sidebar="true" showheader="true" standardcontroller="<Custom Object Name>"
extensions="<Controller Name>">
    <apex:stylesheet value="{!URLFOR($Resource.DSE__bootstrap_sf1, 'bootstrap_sf1/
bootstrap-sf1-0.1.0-beta.15/dist/css/bootstrap-namespaced.css')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery-
ui.css')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery.js')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery-
ui.min.js')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery.blockUI.js')}" />

    <apex:actionStatus id="loadingStatus" onstart="blockDiv('#wrapper');"
onstop="unblockDiv('#wrapper');"/>
    <style>
        .bootstrap-sf1 .form-group { margin-bottom: 3px; color: #000; }
        .bootstrap-sf1 .form-control, .bootstrap-sf1 .form-group{ font-size :12px; }
        .bootstrap-sf1 label{ font-size :11px; color: #4a4a56; }
        .bootstrap-sf1 .form-group input.form-control, .bootstrap-sf1 .form-group
select.form-control{ height: 28px; }
        .bootstrap-sf1 textarea { resize: vertical; }
        .center { text-align: center; }
        .floatRight{ float: right; }
        .zIndex{ z-index: 10000; }
        /*Desktop and Laptops */
        @media only screen and (min-width : 1224px) {
            .right { text-align: right; }
        }
        /*Large Screen Resolution (For Bigger Monitor) */
        @media screen and (min-width : 1824px) {
            .right { text-align: right; }
        }
```

```
    </style>
    <script type="text/javascript">
        var dseJQ = jQuery.noConflict();
        function setFocusOnLoad() {}
        function blockDiv(divId) {
            dseJQ(divId).block({ message: '<br/><apex:image value="/img/loading.gif"/>
Loading', css: { height: '50px', width: '20%' }, overlayCSS: { opacity: .2 } });
        }
        function unblockDiv(divId) {
            dseJQ(divId).unblock();
        }
    </script>

    <div id="wrapper">
        <apex:form id="frm">
            <div class="center">
                <apex:commandbutton value="<Button Label for Edit>" rerender="frm"
action="{!editMode}" rendered="{!NOT(mode)}" status="loadingStatus"/>
                <apex:commandbutton value="<Button Label for Save>" action="{!save}"
rendered="{!mode}" rerender="frm" status="loadingStatus"/>
            </div>
            <div class="bootstrap-sf1">
                <div class="panel panel-default" style="margin-bottom:0px; box-
shadow:none;">
                    <div class="panel-body">
                    <!-- DaaS Fields -->
                        <div class="form-group">
                            <div class="row">
                                <div class="col-lg-6 col-md-6 col-sm-6 col-xs-12">
                                    <apex:outputPanel rendered="{!NOT(mode)}">
                                        <div class="col-lg-4 col-md-4 col-sm-4 col-xs-12
right">
                                            <apex:outputLabel > <DaaS Mapping Name1> </
apex:outputLabel>
                                        </div>
                                    </apex:outputPanel>
                                    <div class="{!wrapperStyle}">
                                        <DSE:DS_DataQuality RecordRef="{!<Variable>}"
FieldGroupName="<DaaS Mapping Name1>" Edit="{!mode}" FormGroupStyle="form-group"
RowStyle="row" LabelStyle="{!labelStyle}" InputStyle="{!inputStyle}" StatusStyle="{!
statusStyle}" FormControlStyle="form-control"> </DSE:DS_DataQuality>
                                    </div>
                                </div>
                                <div class="col-lg-6 col-md-6 col-sm-6 col-xs-12">
                                    <apex:outputPanel rendered="{!NOT(mode)}">
                                        <div class="col-lg-4 col-md-4 col-sm-4 col-xs-12
right">
                                            <apex:outputLabel > <DaaS Mapping Name2> </
apex:outputLabel>
                                        </div>
                                    </apex:outputPanel>
                                    <div class="{!wrapperStyle}">
                                        <DSE:DS_DataQuality RecordRef="{{!<Variable>}}"
FieldGroupName="<DaaS Mapping Name2>" Edit="{!mode}" FormGroupStyle="form-group"
RowStyle="row" LabelStyle="{!labelStyle}" InputStyle="{!inputStyle}" StatusStyle="{!
statusStyle}" FormControlStyle="form-control"> </DSE:DS_DataQuality>
                                    </div>
                                    <div class="col-lg-6 col-md-6 col-sm-6 col-xs-12">
                                        <apex:outputPanel rendered="{!NOT(mode)}">
                                            <div class="col-lg-4 col-md-4 col-sm-4 col-
xs-12 right">
                                                <apex:outputLabel > <DaaS Mapping Name3>
</apex:outputLabel>
                                            </div>
                                        </apex:outputPanel>
                                        <div class="{!wrapperStyle}">
                                            <DSE:DS_DataQuality RecordRef="{{!
<Variable>}}" FieldGroupName="<DaaS Mapping Name3>" Edit="{!mode}" FormGroupStyle="form-
group" RowStyle="row" LabelStyle="{!labelStyle}" InputStyle="{!inputStyle}"
```

```
StatusStyle="{!statusStyle}" FormControlStyle="form-control"> </DSE:DS_DataQuality>
                                                    </div>
                                                </div>
                                            </div>
                                        </div>
                                    </div>
                                </div>
                            </div>
                        </apex:form>
                    </div>
                </apex:page>
```

The Apex code format uses the following parameters:

- `Custom Object Name`. Name of the custom object for which you create the DaaS editor Visualforce page.
- `Controller Name`. Name of the controller that you created for the DaaS editor Visualforce page.
- `DaaS Mapping Name1`, `DaaS Mapping Name2`, and `DaaS Mapping Name3`. Field group names that you specify when you create DaaS mappings.
- `Button Label for Edit`. Label of the button that performs the edit operation.
- `Button Label for Save`. Label of the button that performs the save operation.
- `Variable`. Variable that you use in the Apex code when you create the controller.
- `DS_DataQuality` attributes. You can configure the following attributes:
  - RecordRef. Name of the custom object for which you create the Visualforce page.
  - FieldGroupName. Name of the DaaS mapping used in the Visualforce page.
  - FormGroupStyle. Salesforce bootstrap style for form groups.
  - RowStyle. Salesforce bootstrap style for rows.
  - LabelStyle. Column widths for the labels. You can specify column widths for small, medium, and large screens.
  - InputStyle. Column widths for the input fields. The column widths set the appearance of the input fields on small, medium, and large screens.
  - StatusStyle. Column widths for the verification status. The column widths set the appearance of the verification status on small, medium, and large screens.
  - FormControlStyle. Salesforce bootstrap style for the input fields.
  - Edit. Boolean value that indicates whether the input fields are editable or not. Set to true to make the input fields editable, and set to false to make the input fields non-editable.

6. Click **Save**.

Sample Apex Code

The following sample Apex code creates a Visualforce page named `EmployeePage` and embeds the DaaS components for the field groups named `Employee Phone`, `Employee Address`, and `Employee Email`:

```
<apex:page sidebar="true" showheader="true" standardcontroller="Employee__c"
extensions="EmployeeController">
    <apex:stylesheet value="{!URLFOR($Resource.DSE__bootstrap_sf1, 'bootstrap_sf1/bootstrap-
sf1-0.1.0-beta.15/dist/css/bootstrap-namespaced.css')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery-
ui.css')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery.js')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources, 'jquery-
ui.min.js')}" />
    <apex:includeScript value="{!URLFOR($Resource.DSE__DS_Hierarchy_Resources,
'jquery.blockUI.js')}" />
    <apex:actionStatus id="loadingStatus" onstart="blockDiv('#wrapper');"
```

```
onstop="unblockDiv('#wrapper');"/>
    <style>
           .bootstrap-sf1 .form-group { margin-bottom: 3px; color: #000; }
           .bootstrap-sf1 .form-control, .bootstrap-sf1 .form-group{ font-size :12px; }
           .bootstrap-sf1 label{ font-size :11px; color: #4a4a56; }
           .bootstrap-sf1 .form-group input.form-control, .bootstrap-sf1 .form-group
select.form-control{ height: 28px; }
           .bootstrap-sf1 textarea { resize: vertical; }
           .center { text-align: center; }
           .floatRight{ float: right; }
           .zIndex{ z-index: 10000; }
           /*Desktop and Laptops */
           @media only screen and (min-width : 1224px) {
                .right { text-align: right; }
           }
           /*Large Screen Resolution (For Bigger Monitor) */
           @media screen and (min-width : 1824px) {
                .right { text-align: right; }
           }
   </style>
    <script type="text/javascript">
      var dseJQ = jQuery.noConflict();
            function setFocusOnLoad() {}

            function blockDiv(divId) {
                dseJQ(divId).block({
                    message: '
       <br/>
       <apex:image value="/img/loading.gif"/> Loading',
                    css: { height: '50px', width: '20%' },
                    overlayCSS: { opacity: .2 }
                });
            }

            function unblockDiv(divId) {
                dseJQ(divId).unblock();
            }

    </script>
    <div id="wrapper">
        <apex:form id="frm">
            <div class="center">
                <apex:commandbutton value="Edit" rerender="frm" action="{!editMode}"
rendered="{!NOT(mode)}" status="loadingStatus"/>
                <apex:commandbutton value="Save" action="{!save}" rendered="{!mode}"
rerender="frm" status="loadingStatus"/>
            </div>
            <div class="bootstrap-sf1">
                <div class="panel panel-default" style="margin-bottom:0px; box-shadow:none;">
                    <div class="panel-body">
                        <!-- DaaS Fields -->
                        <div class="form-group">
                            <div class="row">
                                <div class="col-lg-6 col-md-6 col-sm-6 col-xs-12">
                                    <apex:outputPanel rendered="{!NOT(mode)}">
                                        <div class="col-lg-4 col-md-4 col-sm-4 col-xs-12 right">
                                            <apex:outputLabel >Employee Address</
apex:outputLabel>
                                        </div>
                                    </apex:outputPanel>
                                    <div class="{!wrapperStyle}">
                                        <DSE:DS_DataQuality RecordRef="{!a}"
                                 FieldGroupName="Employee Address"
                                 Edit="{!mode}"
                                 FormGroupStyle="form-group"
                                 RowStyle="row"
                                 LabelStyle="{!labelStyle}"
                                 InputStyle="{!inputStyle}"
                                 StatusStyle="{!statusStyle}"
                                 FormControlStyle="form-control"></DSE:DS_DataQuality>
                                    </div>
```

```
                                    </div>
                                    <div class="col-lg-6 col-md-6 col-sm-6 col-xs-12">
                                        <apex:outputPanel rendered="{!NOT(mode)}">
                                            <div class="col-lg-4 col-md-4 col-sm-4 col-xs-12 right">
                                                <apex:outputLabel >Employee Phone</apex:outputLabel>
                                            </div>
                                        </apex:outputPanel>
                                        <div class="{!wrapperStyle}">
                                            <DSE:DS_DataQuality RecordRef="{!a}"
                                     FieldGroupName="Employee
    Phone"
                                     Edit="{!mode}"
                                     FormGroupStyle="form-group"
                                     RowStyle="row"
                                     LabelStyle="{!labelStyle}"
                                     InputStyle="{!inputStyle}"
                                     StatusStyle="{!statusStyle}"
                                     FormControlStyle="form-control"></DSE:DS_DataQuality>
                                        </div>
                                    </div>
                                    <div class="col-lg-6 col-md-6 col-sm-6 col-xs-12">
                                        <apex:outputPanel rendered="{!NOT(mode)}">
                                            <div class="col-lg-4 col-md-4 col-sm-4 col-xs-12 right">
                                                <apex:outputLabel >Employee Email</apex:outputLabel>
                                            </div>
                                        </apex:outputPanel>
                                        <div class="{!wrapperStyle}">
                                            <DSE:DS_DataQuality RecordRef="{!a}"
                                     FieldGroupName="Employee Email"
                                     Edit="{!mode}"
                                     FormGroupStyle="form-group"
                                     RowStyle="row"
                                     LabelStyle="{!labelStyle}"
                                     InputStyle="{!inputStyle}"
                                     StatusStyle="{!statusStyle}"
                                     FormControlStyle="form-control"></DSE:DS_DataQuality>
                                        </div>
                                    </div>
                                </div>
                            </div>
                        </div>
                    </div>
                </apex:form>
            </div>
        </apex:page>
```

Step 5: Override the Standard Salesforce Object Creation Page with Visualforce Page

To enable point of entry verification of contact information for the custom objects, you must override the standard Salesforce object creation page.

1. Select your name and click **Setup**.
2. Under App Setup, select **Create** > **Objects**.

   The **Custom Objects** page appears.
3. Click the custom object.

   The **Custom Object Definition Detail** page appears.
4. In the **Buttons, Links, and Actions** section, select **Edit** for New.

   The **Override Standard Button or Link** page appears.

5. Select **Visualforce Page** and select the appropriate Visualforce page with which you want to override the Salesforce page.

   For example, select the Visualforce page, `EmployeeScoutPage` from the above sample Apex code in step 2.

6. Click **Save**.

Verifying Addresses, Email Addresses, and Phone Numbers

You can verify the addresses, the email addresses, and the phone numbers of custom objects with Informatica Data as a Service. Before you create the Visualforce pages, define a trigger for the custom objects, and build a controller for the Visualforce pages, you must also configure the settings for the contact information verification.

You must configure the address, the email address, and the phone number verification service settings. You must also configure the DaaS mapping for the custom object to verify the contact information of the custom object.

You can verify the contact information when you create records in the custom object creation page. You can also run the Data as a Service (DaaS) batch job to verify the contact information of the existing Salesforce records.

For more information about verifying contact information and configuring settings for the contact information verification, see the chapter *Verifying Addresses, Email Addresses, and Phone Numbers* in *Informatica Cloud Customer 360 Administrator Guide*.

Author

Rajkumari Langlensana
Associate Technical Writer

Acknowledgements

The author would like to thank Gowthamie Balamurugan, Manish Prasad, and Sabitha Pandit for their technical assistance.